- Chapter 5: CPU Scheduling: The Gantt Chart for the schedule, turnaround time, waiting time)

- FCFS
- SJF
- SRTF
- RR

Chapter 8: Deadlocks

- Banker's Algorithm

Consider the following snapshot of a system with 4 types of resources: 9 A, 9 B, 8 C, 6 D. What is the content of the matrix Need? Is the system in a safe state?

Process	Allocation				Max			
	A	B	C	D	A	B	C	D
P1	1	2	0	1	4	4	1	1
P2	1	0	1	1	7	5	6	2
P3	1	1	1	1	3	2	5	1
P4	2	2	0	1	4	2	3	1
P5	0	2	0	1	7	4	5	2

	A	B	C	D
P1	4	2	6	1
P2	5	4	6	2 P1
P3	6	5	7	3 P3
P4	8	7	7	4 P4
P5	8	9	7	5 P5
	9	9	8	6 P2

	A	B	C	D
P1	3	2	1	0
P2	6	5	5	1
P3	2	1	4	0
P4	2	0	3	0
P5	7	2	5	1

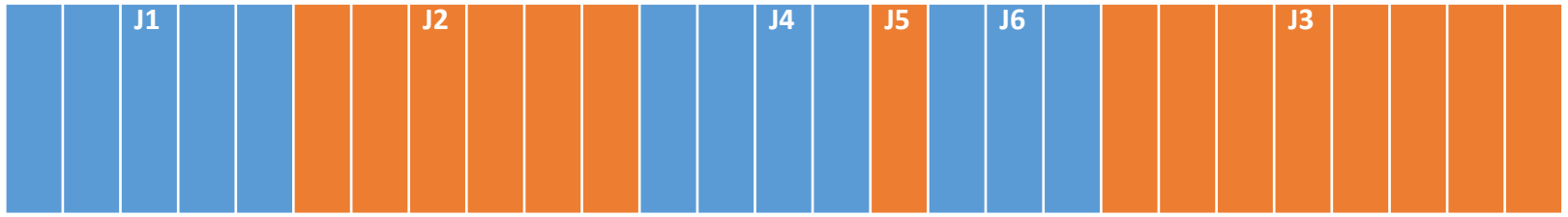
Assume that you have following processes, their arrival times and burst times. For FCFS scheduling algorithms, determine the waiting times for each process and the average waiting times.

Job	Arrival Time	CPU burst
J1	0	5
J2	3	6
J3	4	8
J4	10	4
J5	12	1
J6	15	3

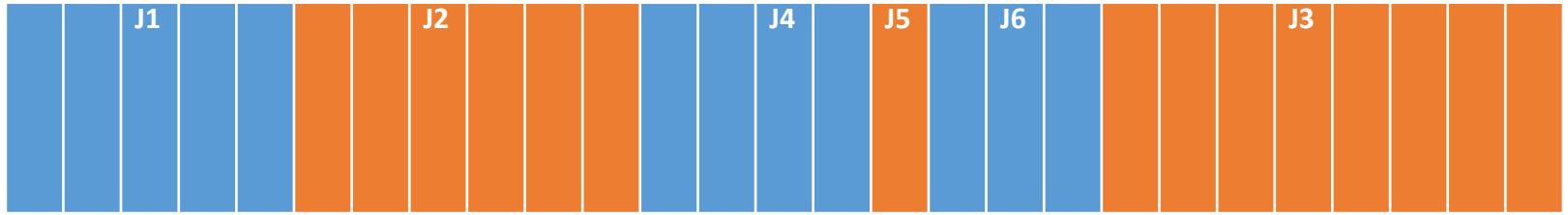


Assume that you have following processes, their arrival times and burst times. For SJF non-preemptive scheduling algorithms, determine the waiting times for each process and the average waiting times.

Job	Arrival Time	CPU burst
J1	0	5
J2	3	6
J3	4	8
J4	10	4
J5	12	1
J6	15	3



SJF(Non-Preemtive)



Job	Thời gian chờ	Thời gian hoàn thành
J1	0	5 – 0 = 5
J2	5 – 3 = 2	11 – 3 = 8
J3	19 – 4 = 15	27 – 4 = 23
J4	11 – 10 = 1	15 – 10 = 5
J5	15 – 12 = 3	16 – 12 = 4
J6	16 – 15 = 1	19 – 15 = 4
Trung Bình	3.67	8.167

Chapter 9: Main Memory

1.

- **First-fit**: Allocate the ***first*** hole that is big enough
- **Best-fit**: Allocate the ***smallest*** hole that is big enough; must search entire list, unless ordered by size
  - Produces the smallest leftover hole
- **Worst-fit**: Allocate the ***largest*** hole; must also search entire list
  - Produces the largest leftover hole.

2.

Consider a logical address space of eight pages of 1024 words each, mapped onto a physical memory of 32 frames.

a. How many bits are there in the logical address?

b. How many bits are there in the physical address?

. Given five memory partitions of 100 KB, 500 KB, 200 KB, 300 KB, and 600 KB (in order), how would each of the First-fit, Best-fit and worst fit place processes of 210 KB, 415 KB, 100 KB, and 586 KB (in order)?

First-fit

	100, 500, 200, 300, 600
P1(210)	100, 290,200, 300, 600
P2(415)	100, 290, 200, 300, 185
P3(100)	0, 290, 200, 300, 185
P4(586)	Wait

Best-fit

	100, 500, 200, 300, 600
P1(210)	100, 500, 200, 90, 600
P2(415)	100, 85, 200, 90, 600
P3(100)	0, 85, 200, 90, 600
P4(586)	0, 85, 200, 90, 14

Worst-fit

	100, 500, 200, 300, 600
P1(210)	100, 500, 200, 300, 390
P2(415)	100, 85, 200, 300, 390
P3(100)	100, 85, 200, 300, 290
P4(586) wait	

Consider the following segment table
What are the physical addresses for the following logical addresses:
0.31; 1.300; 2.10; 3.678; 4.1025

Segment	Base	Limit
0	114	64
1	400	200
2	1200	19
3	1540	800
4	1300	768

0,31
114 + 31 = 145
1,300
Illegal address
2,10
1200 + 10 = 1210
3,678
1540 + 678 = 2218
4,1025
Illegal address

What are the physical addresses in the multi-partition for the following addresses:
115, 256, 2048, 300, 512, 3002, 3000, 6234, 1024, 1111. The base register is 0x2357, the limit register is 5120.

Thanh ghi nền: 0x2357 = 9047 = 2*16^3+3*16^2+5*16+7, thanh ghi giới hạn: 5120
115
9047 + 115 = 9162
256
9047 + 256 = 9303
2048
9047 + 2048 = 11095
300
9047 + 300 = 9347
512
9047 + 512 = 9559
3002
9047 + 3002 = 12049
3000
9047 + 3000 = 12047
6234
Illegal Address
1024
9047 + 1024 = 10071
1111
9047 + 1111 = 10158

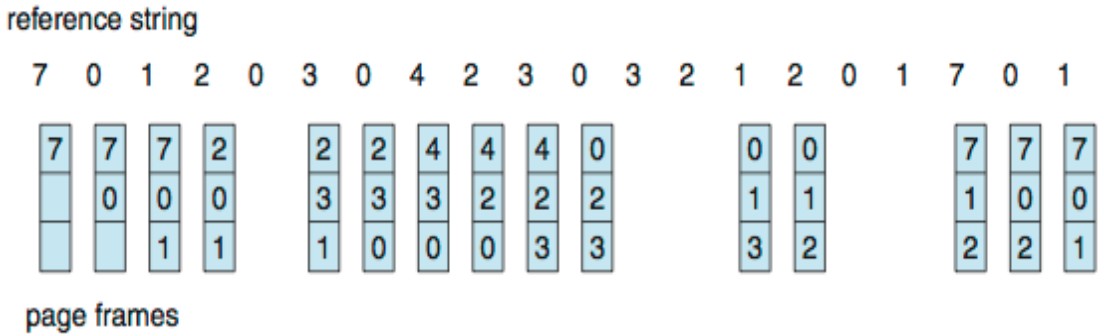
Chapter 10: Virtual Memory

First-In-First-Out (FIFO) Algorithm

- Reference string:
  7,0,1,2,0,3,0,4,2,3,0,3,0,3,2,1,2,0,1,7,0,1
- 3 frames (3 pages can be in memory at a time per process)

First-In-First-Out (FIFO) Algorithm

- Reference string:
  7,0,1,2,0,3,0,4,2,3,0,3,0,3,2,1,2,0,1,7,0,1
- 3 frames (3 pages can be in memory at a time per process)

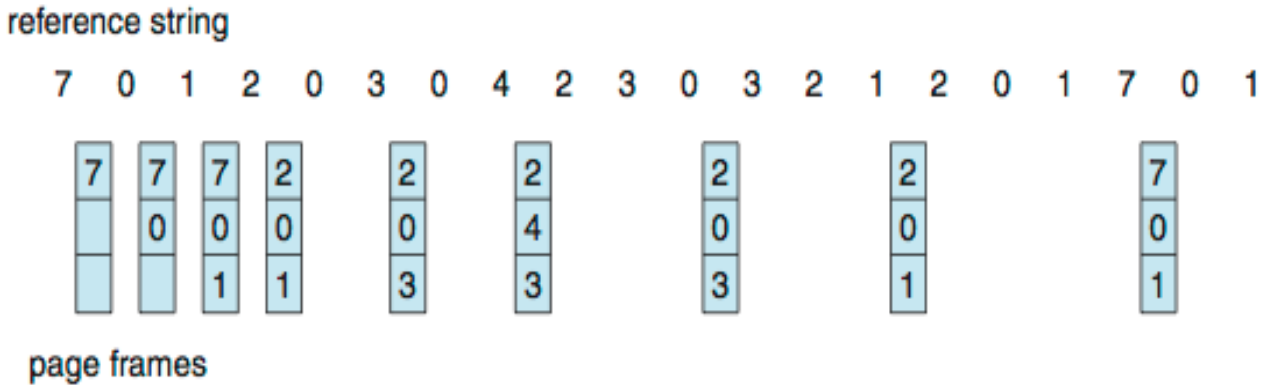


15 page faults

- Can vary by reference string: consider 1,2,3,4,1,2,5,1,2,3,4,5
  - Adding more frames can cause more page faults!
    - Belady's Anomaly
- How to track ages of pages?
  - Just use a FIFO queue

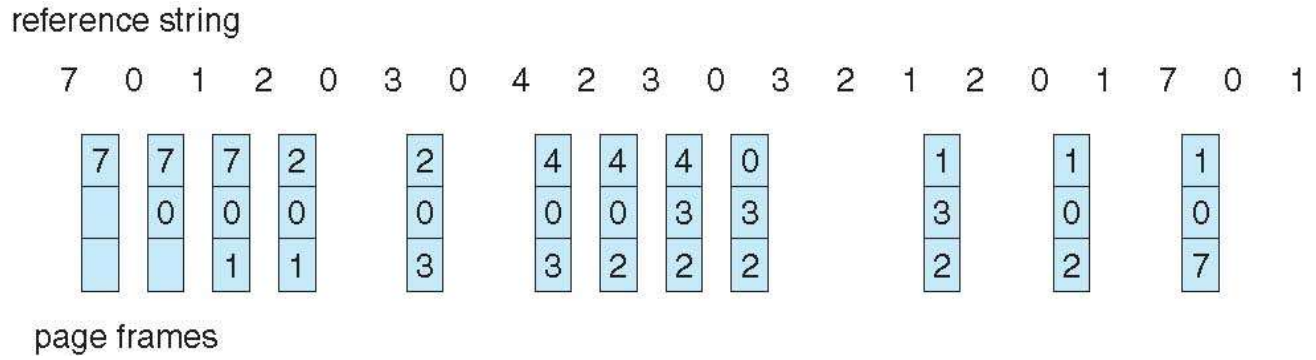
Optimal Algorithm (OPT)

- Replace page that will not be used for longest period of time
  - 9 is optimal for the example
- How do you know this?
  - Can't read the future
- Used for measuring how well your algorithm performs



Least Recently Used (LRU) Algorithm

- Use past knowledge rather than future
- Replace page that has not been used in the most amount of time
- Associate time of last use with each page



- 12 faults – better than FIFO but worse than OPT
- Generally good algorithm and frequently used
- But how to implement?